



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប
មហាវិទ្យាល័យវិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា

មុខវិជ្ជា ៖ Advance C++ Programming
សាស្ត្រាចារ្យ ៖ ស៊ុន ម៉ាឡែន
លេខទូរស័ព្ទ ០១៦៥៨៥៧៨០

<https://elearning.aeu.cloud>



មាតិកា

1. Polymorphism

1.1. Pointer to Base Class

1.2. Abstract Base Class

1.3. ឧទាហរណ៍ និង Coding



Polymorphism

- **Polymorphism (ពហុភាព)** គឺជាកំនិតគ្រឹះដ៏សំខាន់មួយនៅក្នុងការសរសេរកម្មវិធីតាមបែប Object-Oriented Programming (OOP)។ ពាក្យនេះមកពីភាសាក្រិច ដែល "Poly" មានន័យថា ច្រើន (Many) និង "Morph" មានន័យថា ទម្រង់ (Forms)។
- ដូច្នេះ ក្នុងន័យកម្មវិធី **Polymorphism** គឺជាសមត្ថភាពរបស់ Object ផ្សេងៗគ្នាក្នុងការឆ្លើយតបទៅនឹងការហៅ (Message/Function) ដូចគ្នា ប៉ុន្តែផ្តល់លទ្ធផលខុសៗគ្នាទៅតាមលក្ខណៈផ្ទាល់ខ្លួនរបស់ Object នីមួយៗ។

Polymorphism = Overloading + Overriding

Polymorphism

➤ **Function Overriding** គឺជាសមត្ថភាពរបស់ **ខ្លាសកូន (Derived Class)** ក្នុងការសរសេរកូដឡើងវិញ ឬផ្លាស់ប្តូរការអនុវត្ត (Implementation) នៃ Function មួយដែលមានស្រាប់នៅក្នុង **ខ្លាសមេ (Base Class)**។ ដើម្បីធ្វើការ Overriding បាន Function នៅក្នុងខ្លាសកូន ត្រូវតែមាន **ឈ្មោះដូចគ្នា** **ប្រភេទជួរទិន្នន័យ (Return Type) ដូចគ្នា** និង **Parameter ដូចគ្នា** បេះបិទទៅនឹង Function នៅក្នុងខ្លាសមេ ហើយនៅក្នុងភាសា C++ គេត្រូវប្រើពាក្យគន្លឹះ virtual នៅខ្លាសមេ និង override នៅខ្លាសកូន។

Polymorphism

❖ Function Overriding ហេតុអ្វីបានត្រូវប្រើជាមួយ pointer?

➤ ដើម្បីឆ្លើយតបនឹងសំណួរនេះ យើងត្រូវយល់អំពីយន្តការរបស់ C++

ចំនួនពីរគឺ **Static Binding** និងបាតុភូត **Object Slicing**។

➤ នៅក្នុង C++ បើសិនជាយើងបង្កើត Object ធម្មតា (មិនមែនជា Pointer

ទេ) Compiler នឹងប្រើប្រាស់ប្រព័ន្ធ **Static Binding (Early Binding)**។ មាន

ន័យថា ប្រភេទជួរទិន្នន័យ (Type) របស់អថេរនៅពេល Compile ជាអ្វី វា

នឹងហៅ Function របស់ខ្លួននោះជានិច្ច។

Polymorphism

- ផ្ទុយទៅវិញ **Dynamic Binding (Late Binding)** អាចកើតឡើងទៅបាន **លុះត្រាតែ** ប្រើ **Pointer** ឬ **Reference** ប៉ុណ្ណោះ។ Pointer អនុញ្ញាតឱ្យកម្មវិធីពិនិត្យមើល "Object ជាក់ស្តែង" ដែលវាកំពុងចង្អុលទៅនៅពេល Runtime ទើបវាសម្រេចចិត្ត ហៅ Function ត្រឹមត្រូវ។
- ប្រសិនបើអ្នកព្យាយាមយក Object របស់ខ្លួនកូន ទៅដាក់ចូលក្នុង Object របស់ ខ្លួនមេដោយផ្ទាល់ (ដោយមិនប្រើ Pointer) នោះ C++ នឹងធ្វើការ **Slice (កាត់ តម្រឹម)** ទិន្នន័យ។ វាជញ្ជក់យកតែផ្នែកដែលមាននៅក្នុងខ្លួនមេ ហើយបោះ ចោលរាល់លក្ខណៈពិសេស និង Function Overriding របស់ខ្លួនកូនទាំងអស់។

ធ្វើការ Slice (កាត់តម្រឹម) ទិន្នន័យ

```

1  #include <iostream>
2  using namespace std;
3
4  class Animal {
5  public:
6      virtual void sound() { cout << "Animal sound" << endl; }
7  };
8
9  class Dog : public Animal {
10 public:
11     void sound() override { cout << "Woof! Woof!" << endl; }
12 };
13
14 int main() {
15     Dog myDog;
16
17     // ករណីទី ១៖ មិនប្រើ Pointer (កើតមាន Object Slicing)
18     Animal normalAnimal = myDog;
19     normalAnimal.sound(); // លទ្ធផល: "Animal sound" (កូដរបស់កូនត្រូវបានកាត់ចោលបាត់!)
20
21     // ករណីទី ២៖ ប្រើ Pointer (ជៀសវាងការធ្វើ Overriding)
22     Animal* ptrAnimal = &myDog;
23     ptrAnimal->sound(); // លទ្ធផល: "Woof! Woof!" (ហៅកូដរបស់ខ្លួនកូនត្រូវបានត្រឹមត្រូវ)
24
25     return 0;
26 }

```

Animal sound
Woof! Woof!

1.1.Pointer to Base Class

8. ការប្រើប្រាស់ Pointer to Base Class , Abstract Base Class

- 1.១. អ្វីទៅ ហៅថា ការប្រើប្រាស់ Pointer to Base Class ?
- ១.2. អ្វីទៅ ហៅ ថា Abstract Base Class?

Example : Coding .

1.1. Pointer to Base Class

- **Pointer to Base Class** គឺជា Pointer នៃ Base Class (ខ្លួនមេ) ដែលអាចរក្សាទុកអាសយដ្ឋាន (Address) របស់ Derived Class (ខ្លួនកូន) បាន។ វាអនុញ្ញាតឱ្យយើងហៅទៅកាន់ Function របស់ខ្លួនកូន តាមរយៈខ្លួនមេបាន ដោយប្រើប្រាស់ពាក្យគន្លឹះ virtual។ នៅពេលយើងមានខ្លួនកូនច្រើន ដែលស្នង (Inherit) ពីខ្លួនមេតែមួយ យើងអាចប្រើ Pointer របស់ខ្លួនមេតែមួយ ដើម្បីគ្រប់គ្រង Object របស់ខ្លួនកូនទាំងអស់នោះបានយ៉ាងងាយស្រួល។

Syntax:

```
cpp
```

```
BaseClass* pointerName;
```

Coding : Pointer to Base Class

```

1  #include <iostream>
2  using namespace std;
3
4  class Animal {
5  public:
6      // make virtual function to child class become  override
7      virtual void makeSound() {
8          cout << "Animal makes a sound." << endl;
9      }
10 };
11
12 class Dog : public Animal {
13 public:
14     void makeSound() override {
15         cout << "Dog barks: Woof! Woof!" << endl;
16     }
17 };
18
19 int main() {
20     // Pointer of Base Class
21     Animal* myAnimal;
22
23     Dog myDog;
24
25     //give Pointer parents class to Object child class
26     myAnimal = &myDog;
27
28     // call Function Pointer
29     myAnimal->makeSound(); // result: Dog barks: Woof! Woof!
30
31     return 0;
32 }

```

Dog barks:
Woof! Woof!

Coding : Pointer to Base Class

```
1  #include <iostream>
2  using namespace std;
3
4
5  class BankAccount {
6  public:
7      // virtual Override
8      virtual void showInterestRate() {
9          cout << "General Account: Interest rate is 2%." << endl;
10     }
11 };
12
13
14 class SavingsAccount : public BankAccount {
15 public:
16     // Overriding
17     void showInterestRate() override {
18         cout << "Savings Account: Interest rate is 5%." << endl;
19     }
20 };
21
22
23 class FixedDepositAccount : public BankAccount {
24 public:
25     // Overriding
26     void showInterestRate() override {
27         cout << "Fixed Deposit Account: Interest rate is 8%." << endl;
28     }
29 };
30
```

Savings Account:
Interest rate is 5%.
Fixed Deposit
Account: Interest
rate is 8%.

Coding : Pointer to Base Class

```
31  int main() {
32      BankAccount* account; // Pointer
33
34      SavingsAccount savings;
35      FixedDepositAccount fixed;
36
37      // Savings Account
38      account = &savings;
39      account->showInterestRate(); // Savings Account: Interest rate is 5%.
40
41      //Fixed Deposit Account
42      account = &fixed;
43      account->showInterestRate(); // Fixed Deposit Account: Interest rate is 8%.
44
45      return 0;
46  }
```

1.2. Abstract Base Class

- **Abstract Base Class** គឺជាឧបករណ៍ ឬជាវិធីសាស្ត្រចម្បងមួយនៅក្នុងភាសា C++ ដើម្បីយកទ្រឹស្តី Abstraction មកអនុវត្តឱ្យចេញជាលទ្ធផលជាក់ស្តែងនៅក្នុងកូដ។
- **Abstraction** គឺជាគំនិត (Concept) នៃការលាក់បាំងរាល់ព័ត៌មានលម្អិត ឬភាពស្មុគស្មាញនៃប្រព័ន្ធ ក្នុងនោះខាងក្នុង ហើយបង្ហាញតែមុខងារសំខាន់ៗដែលចាំបាច់ទៅកាន់អ្នកប្រើប្រាស់ខាងក្រៅ។
- **Abstract Base Class** គឺជាខ្លួនមេមួយដែលមិនអាចយកទៅបង្កើតជា Object ផ្ទាល់បានទេ (Cannot be instantiated)។ វាត្រូវបានបង្កើតឡើងក្នុងគោលបំណងធ្វើជា "គំរូ ឬប្លង់មេ (Blueprint)" សម្រាប់ឱ្យខ្លួនសក្ខីនៗយកទៅអនុវត្តតាមតែប៉ុណ្ណោះ។
- ខ្លួនមេមួយក្លាយទៅជា Abstract Base Class លុះត្រាតែវាមានយ៉ាងហោចណាស់ Pure Virtual Function មួយ។

1.2. Abstract Base Class

Abstract Base Class ទាក់ទងនឹង Abstraction ដូចម្តេច?

- នៅក្នុង C++ បើអ្នកចង់បង្កើត "ចង្អុល ប្តូរ ឬប្រៀបធៀប" ដូចឧទាហរណ៍ឡាន ខាងលើ អ្នកត្រូវប្រើ Abstract Base Class។ វាដើរតួជា Interface (ផ្ទៃទំនាក់ទំនង) ដែលកំណត់ថា តើ Object នោះអាចធ្វើអ្វីបានខ្លះ (What it does) ប៉ុន្តែវាមិនប្រាប់ថាធ្វើដោយរបៀបណាទេ (How it does)។

Pure Virtual Function គឺជាអ្វី?

- ដឹងទេថាវាជា Function ដែលគ្មានការសរសេរកូដអនុវត្ត (Body) នៅក្នុង ខ្លួនមេទេ ហើយត្រូវបានប្រកាសដោយប្រើសញ្ញា = 0។ រាល់ខ្លួនកូដ ទាំងឡាយណាដែលស្អាតពីវា ត្រូវតែ សរសេរកូដបំពេញ (Override) ឱ្យ Function នោះ ដាច់ខាត។

1.2. Abstract Base Class

Syntax : Abstract Base Class

```
1  class ClassName {  
2  public:  
3      virtual void functionName() = 0; // Pure Virtual Function  
4  };
```

Coding Abstract Base Class(គ)

```
1  #include <iostream>
2  using namespace std;
3
4  // ១. Create Abstract Base Class
5  class Shape {
6  protected:
7      double width, height;
8  public:
9      void setValues(double w, double h) {
10         width = w;
11         height = h;
12     }
13     // Pure Virtual Function
14     virtual double getArea() = 0;
15 };
16
17 // ២. Class 1 (Rectangle)
18 class Rectangle : public Shape {
19 public:
20     double getArea() override {
21         return width * height;
22     }
23 };
24
25 // ៣. Class 2 (Triangle)
26 class Triangle : public Shape {
27 public:
28     double getArea() override {
29         return (width * height) / 2;
30     }
31 };
32
```

Coding Abstract Base Class

```

3  int main() {
4      // Shape s; // error
5
6      // create Pointer of Abstract Base Class
7      Shape* shape1;
8      Shape* shape2;
9
10     Rectangle rect;
11     Triangle tri;
12
13     // Pointer to Object class child
14     shape1 = &rect;
15     shape2 = &tri;
16
17     // set value
18     shape1->setValues(4.0, 5.0);
19     shape2->setValues(4.0, 5.0);
20
21     // using result Pointer parents class
22     cout << "Rectangle Area: " << shape1->getArea() << endl; // result: 20
23     cout << "Triangle Area: " << shape2->getArea() << endl;  // result: 10
24
25     return 0;
26 }

```

Console

I/O



Rectangle Area: 20

Triangle Area: 10

១.៣. ឧទាហរណ៍ ជីវិតពិត

- ដើម្បីងាយយល់ សូមស្រមៃគិតដល់ប្តីតុង "**ចុច (Click)**" តែមួយ៖
- បើអ្នកយកទៅចុចលើប្តីតុង **Play Video** វានឹងចាក់វីដេអូ។
- បើអ្នកយកទៅចុចលើប្តីតុង **Close** វានឹងបិទផ្ទាំង Window។
- បើអ្នកយកទៅចុចលើប្តីតុង **Submit** វានឹងផ្ញើទិន្នន័យ។
- សកម្មភាពគឺតែមួយ (ការចុច) ប៉ុន្តែលទ្ធផលប្រែប្រួលទៅតាម Object ដែលទទួលសកម្មភាពនោះ។

លក្ខណៈ: Function Overloading & Overriding

លក្ខណៈ:	Function Overloading	Function Overriding
ឈ្មោះ: Function	ដូចគ្នា	ដូចគ្នា
Parameter	ត្រូវតែខុសគ្នា	ត្រូវតែដូចគ្នាទាំងស្រុង
ទីតាំង Class	កើតឡើងក្នុង Class តែមួយ	កើតឡើងរវាង Class មេ និង Class កូន (Inheritance)
ពេលវេលាសម្រេច	Compile-time (Static)	Runtime (Dynamic)



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ គ្រប់ទីកន្លែង

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>